

// JavaJub

Собес в МТС Банк

AQA Java

AQA Junior

Вопросы, задачи и подготовка
к live-coding и техническому интервью

Стек

Java • Rest Assured • JUnit 5 • PostgreSQL • Kafka • WireMock • Allure

[// содержание](#)

Содержание

01	Про МТС Банк и формат собеседа	3
02	Стек по вакансии	5
03	Теория тестирования	6
04	Тест-дизайн	8
05	Java Core (Junior)	9
06	HTTP, REST API и Postman	11
07	Rest Assured и WireMock	14
08	JUnit 5, Mockito и тестирование	16
09	SQL (PostgreSQL + Oracle)	18
10	Kafka, RabbitMQ и тестирование	20
11	Безопасность (OWASP)	21
12	Docker, Linux и ELK	22
13	Maven, Git и CI/CD	23
14	Практические задачи	24
15	Пет-проекты для АQA	26
16	План подготовки + чек-лист	27

01 Про МТС Банк и формат собеседа

МТС Банк — цифровой банк экосистемы МТС. Более 7 млн клиентов, 300+ IT-специалистов. Фокус: мобильный банк, кредитование, платежи, интеграции с экосистемой МТС (связь, маркетплейс).

AQA Java (Junior) — инженер автоматизации тестирования. Главный фокус: автотесты для REST API микросервисов, тестирование интеграций через Kafka/RabbitMQ, мок внешних систем через WireMock, SQL-проверки данных. UI-автоматизация (Selenium) НЕ указана в вакансии.

► Структура собеседования (~1.5–2 часа)

Блок	Время	Что проверяют
Знакомство + опыт	~5–10 мин	Пет-проекты, мотивация
Теория тестирования	~15 мин	Виды, уровни, классы эквивалентности
Java Core (Junior)	~15–20 мин	ООП, коллекции, исключения, equals
SQL (Postgres + Oracle)	~15 мин	JOIN + GROUP BY + HAVING
HTTP / REST / Postman	~15 мин	Методы, статусы, JWT, цепочки
Rest Assured + WireMock	~15–20 мин	given/when/then, стаб, verify
Kafka / RabbitMQ (базово)	~5–10 мин	Зачем нужны, как тестировать
Security (специфика банка)	~5–10 мин	OWASP Top 10, SQL-инъекции, IDOR

ФИШКА • Специфика этой вакансии

Rest Assured + CitrusFramework + WireMock — это API + интеграционное тестирование микросервисов с моками внешних систем + тестирование Kafka/RabbitMQ потоков. Selenium/Selenide НЕ упомянуты — это НЕ UI-автоматизация.

ВНИМАНИЕ • Для Junior

В МТС Банке обязательна секция Security Testing — OWASP Top 10, SQL-инъекции, XSS, CSRF, IDOR. Также любят давать практические задачи в Postman с Pre-request Scripts — генерация JWT, цепочка запросов, подпись HMAC.

02 Стек по вакансии

► Основной стек

- Java + ООП — основной язык автотестов
- JUnit 5 / TestNG — тест-фреймворки
- Rest Assured — HTTP API автотесты (given/when/then)
- CitrusFramework — интеграционное тестирование Kafka/RabbitMQ
- WireMock — моки внешних сервисов
- Maven — сборка проекта
- Jackson — сериализация/десериализация JSON
- PostgreSQL + Oracle — SQL для проверки данных
- Kafka + RabbitMQ — брокеры сообщений
- Postman + Swagger/OpenAPI — ручное API-тестирование
- Docker — контейнеризация тестовой среды
- Git — контроль версий
- Linux + bash — работа с логами, серверами
- ELK — поиск по логам (Elasticsearch + Kibana)
- Allure — отчёты по тестам

03 Теория тестирования

Для Junior AQA — это ОБЯЗАТЕЛЬНЫЙ блок. Без теории тестирования не возьмут даже при идеальном Java.

- Верификация vs Валидация.

→ Верификация: «Строим ли мы продукт правильно?» — соответствие требованиям (ревью, инспекции, статический анализ). Валидация: «Строим ли мы правильный продукт?» — соответствие ожиданиям пользователя (тестирование на реальных данных). Классика — путают местами.

- QA vs QC.

→ QA (Quality Assurance): процесс обеспечения качества — стандарты, процессы, предотвращение дефектов. QC (Quality Control): контроль качества — тестирование, нахождение дефектов. QA — шире, QC — часть QA.

- Уровни тестирования.

→ Unit → Integration → System → Acceptance. Unit: отдельный метод/класс (JUnit, Mockito). Integration: взаимодействие компонентов (Testcontainers, WireMock). System: вся система целиком. Acceptance: приёмка пользователем (UAT).

- Smoke / Sanity / Regression / Confirmation — отличия.

→ Smoke: «дымовой тест» — базовая работоспособность после деплоя (5–10 кейсов). Sanity: проверка конкретного фикса/фичи (узко). Regression: проверка, что старое не сломалось (широко, автоматизируем). Confirmation: повторная проверка исправленного бага.

- Black Box / White Box / Grey Box.

→ Black Box: не знаем внутреннюю реализацию, тестируем по спецификации. White Box: знаем код, тестируем ветвления, покрытие. Grey Box: знаем архитектуру, но тестируем как пользователь (AQA — это Grey Box: пишем тесты зная API-контракт и БД-схему).

- Принципы тестирования.

→ Полное тестирование невозможно. Кластеризация дефектов (80% багов в 20% модулей). Парадокс пестицида (одни тесты перестают находить баги — обновляй). Раннее тестирование дешевле. Отсутствие ошибок — заблуждение. Тестирование зависит от контекста.

- Позитивное vs негативное тестирование.

→ Позитивное: проверяем штатное поведение (валидные данные → ожидаемый результат). Негативное: проверяем реакцию на невалидные данные (пустое поле, отрицательная сумма, SQL-инъекция → ожидаем ошибку, а не падение). AQA покрывает оба типа: сначала позитивные (happy path), потом негативные + граничные.

- End-to-End тест — что это?

→ Тест полного пользовательского сценария от начала до конца. Пример: регистрация → логин → создание заявки → одобрение → выдача кредита → погашение. Проходит через все слои: UI/API → бэкенд → БД → Kafka → внешние сервисы. Дороже в поддержке, запускается реже.

- Жизненный цикл бага.

→ New → Open → Assigned → In Progress → Fixed → Resolved → Verified → Closed (или Reopen). Severity: Blocker (блокирует работу), Critical (потеря денег), Major (основной сценарий с workaround), Minor (косметика), Trivial (опечатка). Priority: бизнес-важность исправления.

- Что делать, если разработчик говорит «не баг, а фича»?

→ 1) Проверить требования — ссылка на спецификацию. 2) Если требований нет — эскалировать на РО/аналитика. 3) Оформить баг с чётким ОР/ФР и приложить доказательства. 4) Обсудить на daily/grooming. 5) Если действительно фича — закрыть баг как Won't Fix, но зафиксировать решение.

- Scrum церемонии: Planning, Daily, Review, Retro.

→ *Planning*: что берём в спринт, декомпозиция. *Daily*: 15 мин, три вопроса (что сделал, что буду, что блокирует). *Review*: демо результатов стейкхолдерам. *Retrospective*: что было хорошо, что улучшить, *action items*. QA участвует во всех — на *Planning* оценивает тестируемость, на *Review* демонстрирует автотесты.

04 Тест-дизайн

- **Классы эквивалентности — как применять?**
 - Разбиваем входные данные на группы, в которых поведение системы одинаковое. Пример: поле «возраст» 18–60 → три класса: <18 (отказ), 18–60 (принимаем), >60 (отказ). Из каждого класса берём по одному значению — это и есть минимальный набор тестов.
- **Граничные значения — почему важны?**
 - Баги чаще всего на границах: 0, 1, max, max+1. Пример: пароль 8–20 символов → тестируем: 7 (отказ), 8 (ок), 20 (ок), 21 (отказ). Для Junior: уметь определять границы для любого поля.
- **Тест-кейс vs чек-лист.**
 - Тест-кейс: подробный (шаги, ожидаемый результат, предусловия) — для критичных сценариев, передачи другим. Чек-лист: краткий список проверок без деталей — для опытных, быстрой проверки. В автоматизации: тест-кейс → метод с @Test.
- **State Transition — как использовать?**
 - Для систем с состояниями: заявка на кредит (Новая → На рассмотрении → Одобрена/Отклонена → Выдана). Рисуем диаграмму состояний и переходов. Тестируем: все валидные переходы + невалидные (из «Выдана» нельзя в «Новая»). Пример для банкомата: Idle → Card Inserted → PIN Entered → Menu → ...
- **Severity vs Priority.**
 - Severity: техническая серьёзность (Critical/Major/Minor/Trivial). Priority: бизнес-важность исправления (High/Medium/Low). Пример: Critical + Low = баг критичный, но в третьестепенной фиче (которой пользуются 0.1% клиентов). Minor + High = косметический баг на главной странице.
- **Decision Table — когда применять?**
 - Когда поведение зависит от комбинации условий. Пример: кредит одобряется если (доход > 50к) И (кредитная история хорошая) И (возраст 21–65). Таблица: каждая комбинация условий → действие (одобрить / отклонить / запросить доп. документы). Покрывает все логические ветки.
- **Pairwise testing — зачем?**
 - Когда параметров много (5 полей по 4 значения = 1024 комбинации). Pairwise: покрыть все ПАРЫ параметров (не все комбинации). Инструменты: PICT (Microsoft), AllPairs. Сокращает количество тестов с 1024 до ~20, покрывая 80% багов.
- **Где брать тестовые данные в банке?**
 - Продовые данные нельзя — персональные данные (152-ФЗ). Варианты: 1) Синтетические генераторы (JavaFaker, DataFactory). 2) Маскированный прод (замена ФИО, телефонов, номеров карт). 3) Фикстуры в тестах (@BeforeEach). Для AQA: фикстуры + синтетика. В МТС Банке обязательно маскирование PII.
- **Структура тест-кейса.**
 - ID, заголовок (что тестируем), предусловие (авторизован, баланс > 0), шаги (1. POST /payment..., 2. GET /status...), ожидаемый результат (статус 200, баланс уменьшился), постусловие (откатить данные). В автоматизации: @Test метод = тест-кейс, @BeforeEach = предусловие, assertThat = ожидаемый результат.

05 Java Core (Junior)

- Четыре принципа ООП своими словами.

→ Инкапсуляция: банкомат — не знаешь как хранит деньги, есть методы «снять», «положить». Наследование: электросамокат наследует «транспорт». Полиморфизм: кнопка «play» — разное поведение для видео, музыки, подкаста. Абстракция: руль в машине — не думаешь о рейке и шестерёнках.

- `==` vs `equals` для строк. Почему `String` immutable?

→ `==` сравнивает ссылки (один объект в памяти?). `equals` — содержимое. `String` immutable: безопасность (ключи `HashMap`, передача в файлы/БД), потокобезопасность, кэширование `hashCode`, `String Pool`.

- `final`, `finally`, `finalize` — три разных слова.

→ `final`: класс (нельзя наследовать), метод (нельзя переопределить), поле (нельзя переопределить). `finally`: блок `try-catch-finally`, выполняется ВСЕГДА. `finalize()`: метод `Object`, вызывается GC перед удалением. `Deprecated` с Java 9. Для ресурсов — `try-with-resources`.

- Коллекции: `ArrayList` vs `LinkedList` vs `HashMap`.

→ `ArrayList`: динамический массив, $O(1)$ доступ, $O(n)$ вставка в середину. `LinkedList`: двусвязный список, $O(1)$ вставка/удаление, $O(n)$ доступ. `HashMap`: бакеты + связный список → дерево при ≥ 8 коллизиях. Для Junior: знать когда что использовать.

- Stream API: что такое, промежуточные vs терминальные.

→ `Stream` — конвейер обработки данных. Промежуточные (ленивые): `filter`, `map`, `flatMap`, `sorted`. Терминальные (запускают): `collect`, `forEach`, `count`, `reduce`. Стрим одноразовый.

- `Checked` vs `Unchecked` исключения.

→ `Checked`: от `Exception` (не `RuntimeException`) — компилятор требует обработки. `IOException`, `SQLException`. `Unchecked`: от `RuntimeException` — не обязательно ловить. `NullPointerException`, `IllegalArgumentException`. `try-with-resources`: ресурс реализуем `AutoCloseable`.

- `Overload` vs `Override` — в чём разница?

→ `Overload` (перегрузка): несколько методов с одним именем, но разными параметрами. Компилятор выбирает (статическое связывание). `Override` (переопределение): подкласс меняет реализацию метода родителя. JVM выбирает в рантайме (динамическое связывание, полиморфизм). `Override`: нельзя сузить видимость, нельзя расширить `throws`.

- Абстрактный класс vs интерфейс.

→ Абстрактный класс: состояние (поля), конструкторы, один наследник. Интерфейс: контракт, множественная реализация, Java 8 — `default/static` методы. Когда что: общее поведение + состояние → абстрактный класс. Контракт без реализации → интерфейс. Можно реализовать несколько интерфейсов, но наследовать один класс.

- Generics — зачем нужны?

→ Типобезопасность на этапе компиляции. Без generics: `List list = new ArrayList(); list.add(123); String s = (String)list.get(0)` → `ClassCastException` в рантайме. С generics: `List<String>` — компилятор не даст положить `Integer`. Type erasure: в рантайме параметр стирается (`List<String>` → `List`).

- Integer cache: почему `valueOf(100)==valueOf(100) true`, а `valueOf(200) false`?

→ `IntegerCache` кэширует значения от -128 до 127. `valueOf(100)` возвращает один и тот же объект из кэша → `==` сравнивает ссылки → `true`. `valueOf(200)` создаёт новый объект каждый раз → разные ссылки → `false`. Мораль: для объектов всегда `equals()`, никогда `==`.

- Records в Java — что это?

→ Java 14+: компактный `immutable data`-класс. `record User(String name, int age) {}` — автоматически: конструктор, getters (`name()`, `age()`), `equals`, `hashCode`, `toString`. Нельзя наследовать (`implicitly final`). Можно реализовывать интерфейсы. Идеально для DTO в тестах.

- **Optional — основные методы и антипаттерны.**

→ *isPresent/isEmpty* — проверка. *ifPresent(consumer)* — действие если есть. *orElse(default)* — значение по умолчанию (*eager*). *orElseGet(supplier)* — ленивый. *map/flatMap* — трансформация. Антипаттерн: *optional.isPresent() + optional.get()* → заменить на *map().orElseGet()*. НЕ использовать как поле класса или параметр метода.

- **Модификаторы доступа — порядок.**

→ *private* → (*default/package-private*) → *protected* → *public*. *private*: только в классе. *default*: в пакете. *protected*: в пакете + наследники (даже в другом пакете). *public*: везде. Для AQA: тестируемые методы обычно *public* или *package-private* (тесты в том же пакете).

06 HTTP, REST API и Postman

REST API — главный блок для АQA. HTTP-методы, статусы, идемпотентность, JWT — спрашивают на каждом собеседе.

- HTTP-методы и идемпотентность.
 - GET (получить, идемпотентен), POST (создать, НЕ идемпотентен), PUT (заменить полностью, идемпотентен), PATCH (частично обновить, обычно нет), DELETE (удалить, идемпотентен). Идемпотентность: повторный запрос = тот же результат. POST не идемпотентен → может создать дубль.
- 401 vs 403 — в чём разница?
 - 401 Unauthorized: не аутентифицирован (нет токена или токен невалидный). 403 Forbidden: аутентифицирован, но нет прав (пытаешься удалить чужой заказ). Ловушка: 401 — про identity, 403 — про permissions.
- JWT — структура и как работает?
 - Три части через точку: header.payload.signature. Header: алгоритм (HS256). Payload: данные (userId, exp, roles). Signature: HMAC(header+payload, secret). Хранится на клиенте (localStorage/cookie). Проверяется сервером без БД (stateless). exp — время жизни, после которого нужен refresh token.
- REST vs SOAP.
 - REST: HTTP, JSON, легковесный, stateless, гибкий. SOAP: XML, WSDL, строгий контракт, WS-Security. В банках: REST для новых API, SOAP для legacy (АБС, процессинг). Junior АQA: REST — основной, но упомянуть знание SOAP — плюс.
- Цепочка запросов в Postman.
 - 1) POST /auth/login → получить accessToken. 2) В Tests: pm.environment.set("token", jsonData.accessToken). 3) GET /users/me с заголовком Authorization: Bearer {{token}}. 4) Проверка: pm.expect(pm.response.code).to.eql(200). Pre-request Script: генерация timestamp, подпись HMAC-SHA256.
- Где передавать данные: body / path / query / headers?
 - Path: идентификатор ресурса (/users/123). Query: фильтрация, пагинация (?page=2&sort=name). Body: данные для создания/обновления (POST/PUT/PATCH). Headers: метаданные (Authorization, Content-Type, Accept). НЕ в URL: пароли, токены, персональные данные.
- CORS — что это и когда возникает?
 - Cross-Origin Resource Sharing: браузер блокирует запросы к другому домену. Сервер разрешает через заголовки: Access-Control-Allow-Origin, Access-Control-Allow-Methods. Preflight запрос (OPTIONS) перед POST/PUT. Для АQA: CORS не мешает Rest Assured (нет браузера), но может мешать Postman в браузерной версии.
- PUT vs PATCH — когда что?
 - PUT: замена ресурса целиком. Отправляешь ВСЕ поля, даже неизменённые. Идемпотентен. PATCH: частичное обновление. Отправляешь ТОЛЬКО изменённые поля. Обычно НЕ идемпотентен. Для АQA: тестировать оба — PUT с неполным body → ожидать ошибку или обнуление полей.
- OpenAPI / Swagger — для чего?
 - Спецификация REST API в формате YAML/JSON. Описывает endpoints, методы, параметры, тела запросов/ответов, коды ответов. Swagger UI — интерактивная документация (можно отправлять запросы). API-First: сначала пишем спецификацию, потом код. Для АQA: из OpenAPI можно автогенерировать тесты и модели.
- Pagination: offset/limit vs cursor-based.
 - offset/limit: GET /users?offset=20&limit=10. Простой, но медленный на больших данных (БД пропускает offset строк). cursor-based: GET /users?after=abc123&limit=10. Быстрый (индекс по

cursor), но нельзя прыгнуть на страницу N. Для AQA: тестировать граничные кейсы (offset=0, offset>total, limit=0, limit=max).

- **Content-Type: json vs form-urlencoded vs multipart.**

→ *application/json: JSON в body (REST API). application/x-www-form-urlencoded: пары key=value (формы логина). multipart/form-data: для загрузки файлов (binary + metadata). Для AQA: в Rest Assured — contentType(JSON) для API, contentType("multipart/form-data") для file upload.*

07 Rest Assured и WireMock

Rest Assured — главный фреймворк автоматизации API-тестов. WireMock — для моков внешних сервисов. Оба указаны в вакансии явно.

► Rest Assured: базовые вопросы

- Структура теста Rest Assured.
 - *given()* — настройка запроса (*headers, body, params*). *when()* — HTTP-метод + URL. *then()* — проверки (*statusCode, body*). *extract()* — извлечение данных из ответа. *RequestSpecification* — переиспользование общей конфигурации (*baseUrl, auth, content-type*).
- JSONPath — как извлекать данные?
 - *\$.user.id* — поле *id* объекта *user*. *\$.items[*].name* — все *name* из массива *items*. *\$.items[?(@.price > 100)]* — фильтрация. В Rest Assured: *.extract().path("user.id")*. Для валидации: *body("user.name", equalTo("Alice"))*.
- JSON Schema validation — зачем?
 - Проверяет структуру ответа (типы полей, обязательные поля, формат). *body(matchesJsonSchemaInClasspath("schemas/user.json"))*. Защищает от непредвиденных изменений API-контракта. Генерация схемы: *jsonschema.net* или из *Swagger/OpenAPI*.

► WireMock: базовые вопросы

- WireMock — зачем нужен?
 - Заглушка для внешних сервисов. Вместо реального API курсов валют — мок с фиксированным ответом. Преимущества: тесты не зависят от внешнего сервиса, можно имитировать ошибки (500, таймаут), детерминированные данные.
- Stub vs Verify в WireMock.
 - *Stub*: настроить фиксированный ответ — *stubFor(get("/api/rates").willReturn(okJson(...)))*. *Verify*: проверить, что наш сервис сделал ожидаемый запрос — *verify(getRequestedFor(urlEqualTo("/api/rates")).withHeader("X-API-Key", equalTo("secret")))*. *Scenarios*: стейт-машина — 1-й запрос → 500, 2-й → 200 (мечт retry).

► Практика: тест на PetStore

```
@Test
void getPet_shouldReturnValidData() {
    Pet pet = given()
        .pathParam("petId", 1)
        .when()
        .get("https://petstore.swagger.io/v2/pet/{petId}")
        .then()
        .statusCode(200)
        .extract().as(Pet.class);

    assertThat(pet.getName()).isEqualTo("doggie");
}
```

► Практика: POST + JSON Schema

```
@Test
void createUser_shouldReturn201_andMatchSchema() {
```

```
given()
    .contentType(ContentType.JSON)
    .body(new User("Alice", "alice@mts.ru"))
.when()
    .post("/users")
.then()
    .statusCode(201)
    .header("Location", containsString("/users/"))
    .body(matchesJsonSchemaInClasspath("schemas/user.json"));
}
```

► Практика: WireMock stub + verify

```
// Stub: настроить ответ
stubFor(get(urlEqualTo("/rates/USD"))
    .willReturn(okJson("{\"rate\": 95.50"})));

// Вызвать наш сервис, который ходит в /rates/USD
myService.calculatePayment(...);

// Verify: проверить, что наш сервис сделал запрос
verify(getRequestedFor(urlEqualTo("/rates/USD"))
    .withHeader("X-API-Key", equalTo("secret")));
```

08 JUnit 5, Mockito и тестирование

- JUnit 5: основные аннотации.

→ `@Test` — тестовый метод. `@BeforeEach/@AfterEach` — до/после каждого теста. `@BeforeAll/@AfterAll` — до/после всех (static). `@DisplayName` — читаемое имя. `@Disabled` — пропустить. `@Tag` — категория (smoke, regression). `@ParameterizedTest` — параметризация.

- Параметризация тестов в JUnit 5.

→ `@ParameterizedTest` + источник данных: `@ValueSource(strings = {"a", "b"})`, `@CsvSource({"1, true", "2, false"})`, `@MethodSource("dataProvider")`, `@CsvFileSource(resources = "/data.csv")`. Для AQA: параметризация status-кодов, валидных/невалидных данных, endpoint'ов.

- Hard assertions vs Soft assertions.

→ Hard (по умолчанию): при первом `assertFalse` тест падает, остальные проверки не выполняются. Soft (`assertAll` в JUnit 5, `SoftAssertions` в AssertJ): все проверки выполняются, ошибки собираются. Для AQA: Soft assertions для проверки нескольких полей ответа — увидеть ВСЕ ошибки за один запуск.

- Mockito: Mock vs Spy — в чём разница?

→ Mock: полностью пустой объект, все методы возвращают default (null, 0, false). Spy: обёртка над реальным объектом, методы вызываются по-настоящему, если не замочаны. Для Spy: `doReturn().when(spy).method()` (не `when(spy.method())` — иначе вызовется реальный метод).

- ArgumentCaptor — зачем нужен?

→ Захват аргумента, переданного в замочанный метод. `verify(service).send(captor.capture());` `assertThat(captor.getValue().getEmail()).isEqualTo("test@mts.ru")`. Для AQA: проверить, что сервис отправляет правильные данные во внешний API.

- Testcontainers — что это?

→ Библиотека для запуска Docker-контейнеров из Java-тестов. `PostgreSQLContainer`, `KafkaContainer`, `GenericContainer` (WireMock). `@Testcontainers` + `@Container`. Для AQA: поднять реальную БД вместо H2, реальный Kafka вместо embedded. Reusable containers для ускорения.

- @SpringBootTest vs @WebMvcTest vs @DataJpaTest.

→ `@SpringBootTest`: поднимает ВСЕ контекст (медленно, для E2E). `@WebMvcTest`: только контроллеры + `MockMvc` (без БД). `@DataJpaTest`: только JPA-слой + встроенная БД. Для AQA: `@SpringBootTest` + `Testcontainers` для полных интеграционных тестов.

- Что такое фикстура в тестах?

→ Подготовка данных перед тестом. `@BeforeEach`: создать пользователя, залить данные в БД. `@AfterEach`: удалить данные. Builder/Factory для создания тестовых объектов. Фикстуры должны быть независимыми — тесты не должны зависеть от порядка выполнения.

09 SQL (PostgreSQL + Oracle)

SQL обязателен — проверяете данные в БД после API-запросов. В стеке обе БД: PostgreSQL и Oracle.

- DML / DDL / DCL / TCL.
 - DML: *SELECT, INSERT, UPDATE, DELETE* (данные). DDL: *CREATE, ALTER, DROP, TRUNCATE* (структура). DCL: *GRANT, REVOKE* (права). TCL: *COMMIT, ROLLBACK, SAVEPOINT* (транзакции).
- NULL в SQL — почему NULL = NULL это UNKNOWN?
 - NULL — отсутствие значения, не ноль. NULL = NULL → UNKNOWN (не true и не false). Для проверки: *IS NULL / IS NOT NULL*. *COALESCE(a, b)* — первый не-NULL. Ловушка: *WHERE status != 'active'* НЕ вернёт строки с NULL-статусом.
- PostgreSQL vs Oracle — ключевые отличия.
 - PG: *SERIAL/IDENTITY* (автоинкремент), *LIMIT/OFFSET*, нет *Read Uncommitted*. Oracle: *SEQUENCE, ROWNUM / FETCH FIRST N ROWS, NVL* вместо *COALESCE*, *DUAL* для *SELECT* без таблицы, *CONNECT BY* для иерархий. В PG *Repeatable Read* решает *phantom reads (MVCC)*.
- Порядок выполнения SQL-запроса.
 - *FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT*. Важно: *WHERE* выполняется ДО *GROUP BY* (нельзя фильтровать по агрегату в *WHERE*). *HAVING* — после *GROUP BY* (для агрегатов).
- Подзапрос vs JOIN vs CTE — когда что?
 - Подзапрос: в *WHERE* для фильтрации (*WHERE id IN (SELECT ...)*). *JOIN*: для объединения данных из нескольких таблиц. *CTE (WITH ... AS)*: для читаемости сложных запросов, рекурсия. Коррелированный подзапрос: выполняется для каждой строки внешнего запроса (медленно). Для AQA: *CTE* упрощает отладку — можно запустить каждую часть отдельно.
- UNION vs UNION ALL.
 - *UNION*: объединяет результаты и удаляет дубликаты (медленнее — сортировка). *UNION ALL*: объединяет без удаления (быстрее). Для AQA: если знаешь что дубликатов нет — *UNION ALL*.
- VIEW vs MATERIALIZED VIEW.
 - *VIEW*: виртуальная таблица, запрос выполняется при каждом обращении. *MATERIALIZED VIEW*: результат сохраняется на диске, нужен *REFRESH* для обновления. Для AQA: *VIEW* для упрощения запросов в тестах, *MATERIALIZED VIEW* для отчётов.
- PRIMARY KEY vs UNIQUE — отличия.
 - *PRIMARY KEY*: уникальность + *NOT NULL* + один на таблицу + кластерный индекс. *UNIQUE*: уникальность (допускает один NULL) + может быть несколько. *Foreign Key* ссылается на *PRIMARY KEY* (или *UNIQUE*).
- CASE WHEN — для условных значений.
 - *SELECT name, CASE WHEN balance > 100000 THEN 'VIP' WHEN balance > 10000 THEN 'Standard' ELSE 'Basic' END AS category FROM accounts*. Для AQA: *CASE WHEN* удобен в SQL-проверках — классифицировать данные без Java-логики.

► SQL-задачи

Клиенты с >5 транзакциями за месяц

```
SELECT c.name, COUNT(t.id) AS cnt
FROM clients c
JOIN accounts a ON a.client_id = c.id
JOIN transactions t ON t.account_id = a.id
WHERE t.created_at >= NOW() - INTERVAL '1 month'
```

```
GROUP BY c.id, c.name  
HAVING COUNT(t.id) > 5;
```

Клиенты без транзакций (LEFT JOIN + IS NULL)

```
SELECT c.* FROM clients c  
LEFT JOIN accounts a ON a.client_id = c.id  
LEFT JOIN transactions t ON t.account_id = a.id  
WHERE t.id IS NULL;
```


10 Kafka, RabbitMQ и тестирование очередей

- Зачем брокеры в банке?

→ Асинхронные платежи (не блокировать пользователя), уведомления (SMS/push), события между микросервисами (заявка одобрена → выдать кредит → отправить SMS). Kafka: pull, хранит сообщения, high throughput. RabbitMQ: push, удаляет после ACK, rich routing.

- Как тестировать Kafka в автотесте?

→ 1) Поднять Kafka через Testcontainers. 2) Отправить сообщение `KafkaTemplate.send(topic, message)`. 3) Прочитать ответное через `KafkaConsumer.poll()`. 4) Проверить содержимое. Через Citrus: `send(kafka).message(...) → receive(kafka).message(...).validate(...)`. Awaitility для ожидания: `await().atMost(5, SECONDS).until(() -> ...)`.

- Eventual consistency — почему `assertEquals` сразу может упасть?

→ Kafka/RabbitMQ — асинхронные. Сообщение отправлено, но ещё не обработано consumer'ом. `assertEquals` сразу после `send` → данных в БД ещё нет. Решение: Awaitility с polling, или `Thread.sleep` (плохо — хрупко), или `callback/listener`.

- ACK / NACK / DLQ — что это?

→ ACK: consumer подтверждает обработку. NACK: отказ (retry или DLQ). DLQ (Dead Letter Queue): очередь для «мёртвых» сообщений, которые не удалось обработать после N ретраев. Тестировать DLQ: отправить невалидное сообщение → проверить, что попало в DLQ.

11 Безопасность (OWASP)

В МТС Банке Security Testing обязателен даже для Junior AQA. OWASP Top 10, SQL-инъекции, XSS, CSRF, IDOR — знать на уровне «что это и как проверить».

- SQL-инъекция — что это и как проверить?
 - Вставка SQL-кода в поле ввода: ' OR 1=1 -- в поле логина. Если приложение подставляет ввод напрямую в SQL — получаем доступ ко всем данным. Как проверить: ввести ' ; DROP TABLE users; -- в поля формы, проверить ответ. Защита: Prepared Statements (параметризованные запросы).
- XSS — что это и как проверить?
 - Cross-Site Scripting: вставка JavaScript в данные. <script>alert(1)</script> в поле имени → если отобразится без экранирования — XSS. Stored XSS: скрипт сохраняется в БД. Reflected: в URL-параметре. Защита: экранирование вывода, Content-Security-Policy.
- IDOR — что это?
 - Insecure Direct Object Reference: один клиент видит данные другого. GET /api/users/123/accounts — если проверки прав нет, можно подставить чужой ID и увидеть чужие счета. Как тестировать: авторизоваться user A, попытаться GET ресурс user B → должен быть 403.
- Маскирование PII в логах.
 - PII (Personal Identifiable Information): номер карты, паспорт, телефон. В логах нельзя в открытом виде. Проверка: сделать запрос с номером карты → посмотреть логи → должно быть ****1234, не полный номер. PCI DSS: нельзя хранить CVV вообще.
- Тестирование авторизации.
 - Без токена → 401. С истёкшим токеном → 401. С токеном другого пользователя → 403. С битым токеном → 401. С правильным токеном → 200. Double submit: дважды нажать «Оплатить» → должно списать один раз (идемпотентность, Idempotency-Key).
- CSRF — что это и как проверить?
 - Cross-Site Request Forgery: злоумышленник заставляет браузер отправить запрос от имени авторизованного пользователя. Пример: скрытая форма на вредоносном сайте делает POST /transfer. Защита: CSRF-токен в форме/заголовке, SameSite cookie. Для AQA: попробовать запрос без CSRF-токена → должен быть 403.
- PCI DSS — что должен знать AQA?
 - Payment Card Industry Data Security Standard. Нельзя хранить CVV/CVC вообще. Номер карты только маскированный (****1234). Логи не должны содержать полный PAN. AQA проверяет: отправить карту → посмотреть в логах/БД → должна быть маскирована. Тест: GET /cards → в ответе только ****1234.

12 Docker, Linux и ELK

- Docker: образ vs контейнер. CMD vs ENTRYPOINT.

→ Образ: шаблон (класс). Контейнер: запущенный экземпляр (объект). CMD: аргументы по умолчанию (легко переопределить). ENTRYPOINT: основная команда (сложнее). docker-compose: несколько контейнеров (Postgres + Kafka + сервис). Testcontainers: Docker из Java-кода для тестов.

- Linux: найти все ERROR за сегодня и посчитать.

→ `grep "$(date +%Y-%m-%d)" app.log | grep -c "ERROR". find: find . -name "*.log" -mtime -1 (файлы за сутки). Уникальные IP: grep -oE '([0-9]{1,3}\.){3}[0-9]{1,3}' access.log | sort -u. tail -f app.log | grep ERROR — мониторинг в реальном времени.`

- ELK: как искать логи теста?

→ Elasticsearch хранит/индексирует, Kibana — UI для поиска. Фильтр: `traceld` + временной диапазон. KQL: `level:ERROR AND service:payment`. MDC: Mapped Diagnostic Context — `traceld` пробрасывается через потоки. К базу прикладываешь: `запрос/ответ` + лог по `traceld` + `timestamp`.

- Allure: зачем и основные аннотации?

→ Наглядные отчёты с историей, вложениями, ретраями. `@Step` — шаг теста в отчёте. `@Description` — описание. `@Severity` — приоритет. `@Epic/@Feature/@Story` — иерархия. `Allure.addAttachment` — прикрепить `запрос/ответ/скриншот`.

13 Maven, Git и CI/CD

- Жизненный цикл Maven.

→ *validate* → *compile* → *test* → *package* → *verify* → *install* → *deploy*. *mvn test*: прогоняет unit-тесты (*Surefire plugin*). *mvn verify*: прогоняет интеграционные (*Failsafe plugin*). *Scope*: *compile* (по умолчанию), *test* (только для тестов), *provided* (контейнер предоставим), *runtime*.

- Surefire vs Failsafe — разница.

→ *Surefire*: запускает unit-тесты (**Test.java*) на фазе *test*. *Failsafe*: запускает интеграционные (**IT.java*) на фазе *verify*. *Failsafe* НЕ падает сразу при ошибке — собирает все результаты, потом *fail*. Для AQA: API-автотесты — через *Failsafe* (интеграционные).

- Merge vs Rebase в Git.

→ *Merge*: создаёт коммит-слияние, история ветвится (видно откуда пришли изменения). *Rebase*: перемещает коммиты на вершину целевой ветки, линейная история. *Rebase* в публичных ветках — не стоит (переписывает историю). Для AQA: *merge* для *feature*-веток, *rebase* для синхронизации с *main*.

- Как автотесты встраиваются в CI/CD pipeline?

→ 1) *Smoke*-тесты: после каждого деплоя (быстрые, 5–10 кейсов). 2) Регрессия: на *merge request* или ночной запуск (полный набор). 3) *Nightly*: полная регрессия + отчёт *Allure*. *Jenkins/GitLab CI* → *mvn verify* → *Allure report*. Если тест упал в CI, но проходит локально: логи *Jenkins*, разница окружений, таймауты, *race condition*.

14 Практические задачи (live-coding)

В МТС Банке live-coding на 30–45 минут. Junior-уровень: палиндром, анаграмма, дубликаты, FizzBuzz. Готовиться — написать каждую без IDE за 5–10 минут.

► Задача 1. Палиндром

```
public boolean isPalindrome(String s) {
    String clean = s.replaceAll("[^a-zA-Zа-яА-Я]", "").toLowerCase();
    return clean.equals(new StringBuilder(clean).reverse().toString());
}
// "А роза упала на лапу Азора" → true
```

► Задача 2. Подсчёт символов

```
public Map<Character, Integer> charCount(String s) {
    Map<Character, Integer> map = new HashMap<>();
    for (char c : s.toCharArray()) {
        map.merge(c, 1, Integer::sum);
    }
    return map;
}
// "banana" → {b=1, a=3, n=2}
```

► Задача 3. Найти дубликаты

```
public Set<Integer> findDuplicates(int[] arr) {
    Set<Integer> seen = new HashSet<>();
    Set<Integer> dups = new HashSet<>();
    for (int n : arr) {
        if (!seen.add(n)) dups.add(n);
    }
    return dups;
}
```

► Задача 4. Парсинг строки в Map

```
public Map<String, String> parse(String s) {
    // "name=Иван;age=30;city=Москва"
    return Arrays.stream(s.split(";"))
        .map(pair -> pair.split("=", 2))
        .collect(Collectors.toMap(a -> a[0], a -> a[1]));
}
```

► Задача 5. Тест-дизайн: форма перевода денег

- Позитивные: валидная сумма, валидные счета → 200
- Граничные: сумма 0, 0.01, максимум, максимум+0.01
- Негативные: отрицательная сумма, нечисловые символы, пустые поля
- Перевод самому себе → ожидаемое поведение?
- Двойное нажатие «Отправить» → списать один раз (идемпотентность)

- Недостаточный баланс → 400 + сообщение
- Параллельные переводы с одного счёта → race condition
- Безопасность: перевод с чужого счёта (IDOR) → 403, CSRF
- Логи: номер карты замаскирован (****1234)

15 Pet-проекты для AQA Junior

Pet-проект для AQA — это автотесты к публичному API. Покажите структуру проекта, чистоту кода, отчёты.

► Проект 1. Автотесты для PetStore API

- Java + JUnit 5 + Rest Assured + Allure
- CRUD-тесты: POST /pet → GET → PUT → DELETE
- Позитивные + негативные + граничные кейсы
- RequestSpecification для переиспользования
- JSON Schema validation
- Maven + Allure отчёт
- README с инструкцией запуска
- GitHub Actions: при push прогоняются тесты

► Проект 2. API-тесты + WireMock + Testcontainers

- Spring Boot сервис + PostgreSQL + WireMock
- Тесты через Rest Assured + Testcontainers (PG в Docker)
- WireMock: мок внешнего сервиса курсов валют
- Allure отчёт с прикреплёнными запросами/ответами
- Проверка данных в БД через JdbcTemplate после API-вызова

► Проект 3. Тестирование Kafka через Citrus

- Spring Boot + Kafka + PostgreSQL
- Citrus или Spring Kafka Test: отправка в топик → чтение результата
- Awaitility для ожидания асинхронного результата
- Testcontainers для Kafka и PostgreSQL
- Документация: описание сценариев, как запустить

СОВЕТ • Что оценивают в pet-проекте AQA

Структура проекта (Maven-конвенция). Нет дубликатов (RequestSpecification). Нет хардкода URL (переменные окружения). Тесты независимы друг от друга. README с инструкциями. Покрытие: позитивные + негативные + граничные. Allure-отчёт.

16 План подготовки + чек-лист

► За 2–3 недели

- Теория тестирования: уровни, виды, классы эквивалентности, граничные значения
- Java Core: ООП, коллекции, исключения, Stream API — Junior уровень
- Rest Assured: given/when/then, JSONPath, JSON Schema, RequestSpec
- SQL: JOIN, GROUP BY, HAVING, оконные функции — Postgres + Oracle
- Postman: цепочки запросов, Pre-request Script, Tests
- Pet-проект: автотесты для PetStore API + Allure + GitHub
- OWASP Top 10: SQL-инъекции, XSS, CSRF, IDOR

► За неделю

- 1–2 мок-интервью
- Написать баг-репорт за 5 минут по описанию
- Тест-дизайн: форма логина, перевод денег, банкомат
- Linux: grep, find, tail -f, простой bash-скрипт
- WireMock: stub + verify + scenarios (retry)
- Все вопросы из гайда ВСЛУХ

► В день собеседа

- Камера, микрофон, IDE наготове
- Рассуждать вслух — молчание хуже «дай подумать»
- Тест-дизайн: начинай с позитивных → граничных → негативных → security
- SQL: FROM → JOIN → WHERE → GROUP BY → HAVING → ORDER BY
- Упомяни CitrusFramework — сильный плюс для этой вакансии
- 2–3 вопроса: команда, проект, процессы, менторство

ВНИМАНИЕ • AQA ≠ Backend

Для AQA Junior главное: теория тестирования + тест-дизайн + Rest Assured + SQL + HTTP. Java Core — на Junior уровне (без многопоточности и JMM). Покажите системность мышления при тест-дизайне — это важнее, чем глубокое знание Java.

► Финальный чек-лист

Блок	Готов, если можешь...
Теория	Верификация vs Валидация + Smoke/Sanity/Regression + принципы
Тест-дизайн	Классы эквивалентности + граничные + State Transition + баг-репорт
Java Core	ООП 4 принципа + equals/hashCode + коллекции + Stream API

HTTP/REST	Методы + идиempotentность + 401 vs 403 + JWT + CORS
Postman	Цепочка запросов + Pre-request Script + переменные окружения
Rest Assured	given/when/then + JSONPath + JSON Schema + RequestSpec
WireMock	Stub + Verify + Scenarios (retry) + задержки
SQL	JOIN 3 таблиц + GROUP BY + HAVING + NULL + PG vs Oracle
Kafka/RabbitMQ	Push vs Pull + ACK/NACK/DLQ + Awaitility + Testcontainers
Security	SQL-инъекция + XSS + IDOR + маскирование PII + идиempotentность
Docker/Linux	docker-compose + grep/find + tail -f + chmod
Pet-проект	Автотесты PetStore + Allure + README + GitHub Actions

Удачи на собесе!

// git push origin offer

JavaJub